DHBW Stuttgart

Karsten Keßler

Datenbanksysteme

4. Relationales Modell

Kurs Informatik TINF24ITA

Stand: 01.10.2026

Inhalte der heutigen Vorlesung

- **Das Relationale Modell**: Relation, Schema, Instanz, Domänen
- **Schlüssel**: Kandidaten-, Super- und Primärschlüssel
- **Integritätsbedingungen**: lokal, global, referenziell; Löschregeln
- **Kardinalitäten**: (min, max)- und MC-Notation
- **Überführung ER-Modell → Relationenmodell**: hierarchische, konditionelle, netzwerkförmige und rekursive Beziehungen
- **Spezialisierung und Generalisierung**: Umsetzung von Obertyp und Subtypen

Eingangsbeispiel

Die Grundidee relationaler Datenbanken ist es, dass sämtliche Daten in Tabellen vorliegen. Diese bestehen aus Zeilen und Spalten.

Jede Spalte muss einen Spaltennamen und einen Datentyp besitzen (evtl. Zustand NULL zulässig)

Jede Zeile (Datensatz, Item, Record, Tupel) muss durch einen Primärschlüssel eindeutig identifiziert werden. Dieser kann eine fortlaufende Nummer oder eine UUID sein und wird oft „ID-Spalte“ genannt.

Daten können auf mehrere miteinander verknüpfte Tabellen verteilt werden.

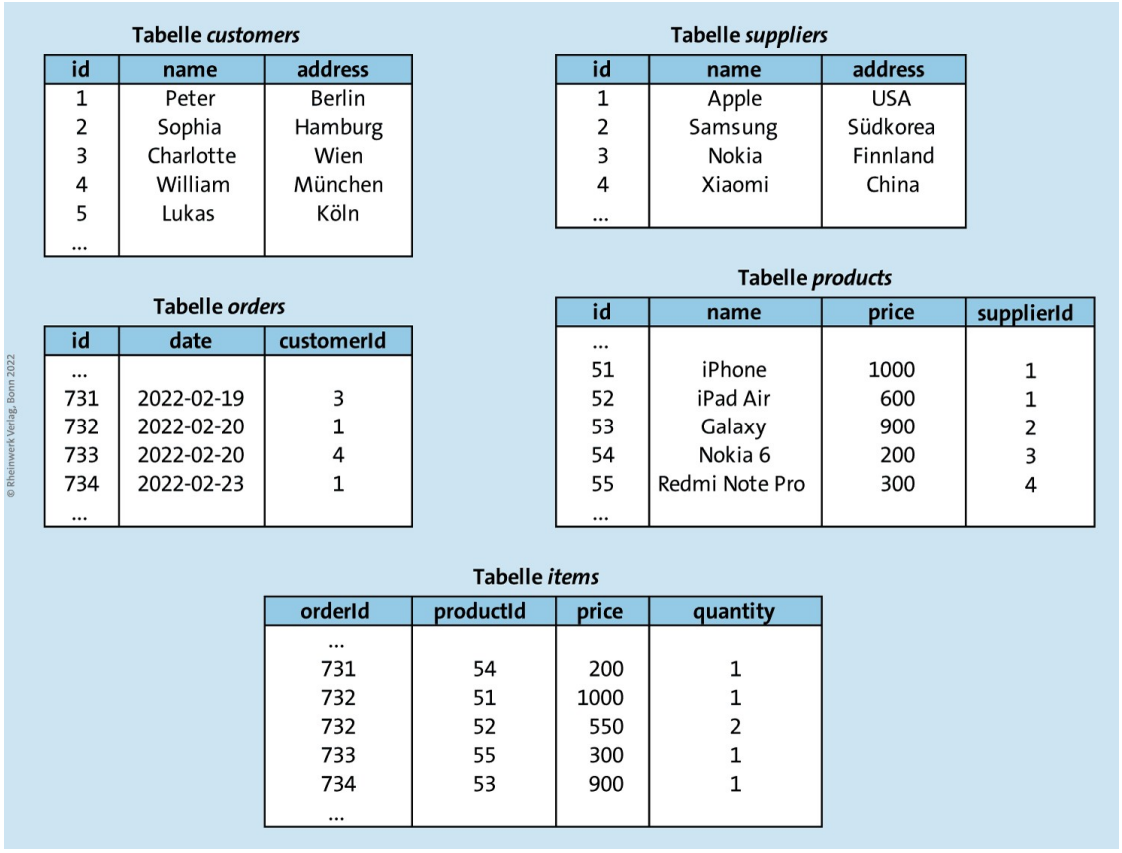


Tabelle *customers*

id	name	address
1	Peter	Berlin
2	Sophia	Hamburg
3	Charlotte	Wien
4	William	München
5	Lukas	Köln
...		

Tabelle *suppliers*

id	name	address
1	Apple	USA
2	Samsung	Südkorea
3	Nokia	Finnland
4	Xiaomi	China
...		

Tabelle *orders*

id	date	customerId
...		
731	2022-02-19	3
732	2022-02-20	1
733	2022-02-20	4
734	2022-02-23	1
...		

Tabelle *products*

id	name	price	supplierId
...			
51	iPhone	1000	1
52	iPad Air	600	1
53	Galaxy	900	2
54	Nokia 6	200	3
55	Redmi Note Pro	300	4
...			

Tabelle *items*

orderId	productId	price	quantity
...			
731	54	200	1
732	51	1000	1
732	52	550	2
733	55	300	1
734	53	900	1
...			

© Rheinwerk Verlag, Bonn 2022

Hinführung

- E. F. Codd, 1970, Turing-Award-Gewinner
- Am weitesten verbreitet, praxisbewährt, immer noch nicht abgelöst
- Ablösung der hierarchischen und Netzwerk-Modelle

Relation **BUCH**

INV_NR	TITEL	ISBN	AUTOR
0001	Kommunikation im Netz	3-123	D. Reichardt
0002	Learn Java Now	3-239	S. Davis
0003	JavaScript	3-111	D. Flanagan
…	…	…	…
0196	Datenbanken	3-826	A. Heuer

Attribute (Spalten)

Tupel (Zeilen)
ein Datensatz je Buch

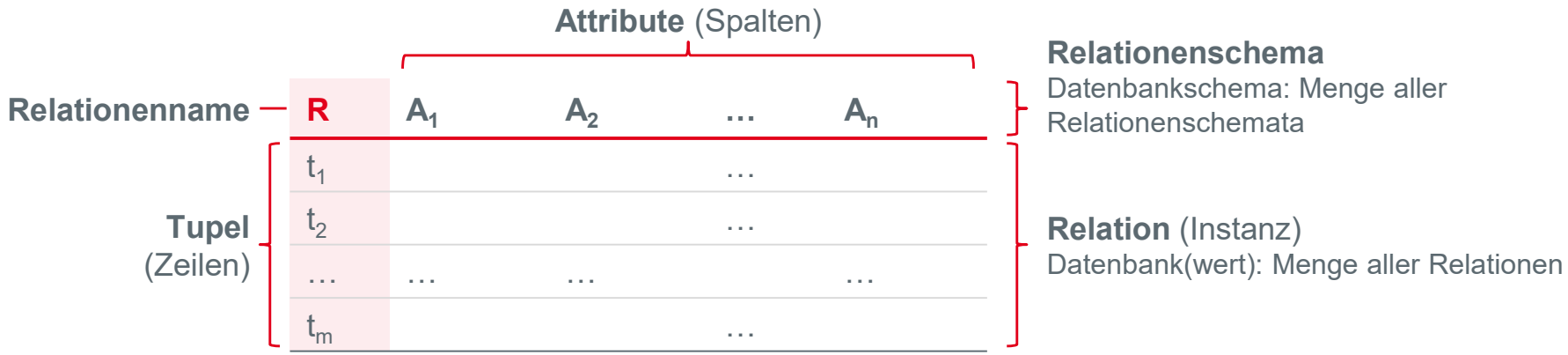
☐ **Primärschlüssel** INV_NR: identifiziert jedes Tupel eindeutig

Aufbau und Funktionen: Relationenmodell

Eine relationale Datenbank enthält eine Menge von **Relationen**.

Eine Relation *R* besteht aus zwei Bestandteilen:

- **Schema**: legt die Struktur fest, also den Namen der Relation sowie Namen und Datentypen der Spalten
- **Instanz**: der aktuelle Inhalt, eine Tabelle mit Zeilen und Spalten



Domänen und formale Definition

- Beispiel: **Telefonbuch** mit den Attributen Name, Straße und Telefonnummer
- Jedem Attribut ist ein Wertebereich zugeordnet, die **Domäne**
  - D_{Name} = string, $D_{Straße}$ = string, $D_{Telefonnummer}$ = string
  - Schema: Telefonbuch (Name: string, Straße: string, Telefonnummer: string)
- Eine **Relation** (Instanz) ist eine Untermenge des kartesischen Produkts der Domänen
  - $R \subseteq D_1 \times D_2 \times \ldots \times D_n$
  - Beispiel: Buch $\subseteq D_{ISBN} \times D_{Titel} \times D_{Autor}$ = string × string × string
- Ein **Relationenschema** ist eine Untermenge der im Universum vorhandenen Attribute: **R** = $\{A_1, \ldots, A_n\}$

Relation **Telefonbuch**

Name	Straße	Telefonnummer
Cantor	Zahlengerade	987654
Jacobi	Matrizenweg	135791
Riemann	Integralpfad	246810

Formale Spezifikation

Universum U	Die (endliche) Menge aller Attribute, die in einem Datenbankzustand vorkommen können
Relationenschema	$R \subseteq U$ mit $R = \{A_1, \ldots, A_n\}$
Datenbankschema	$S = \{R_1, \ldots, R_m\}$
Identifizierende Attributmenge	Eine Menge $K \subseteq R$, sodass K jedes Tupel einer Relation über R eindeutig identifiziert
Schlüssel	Eine bezüglich $\subseteq$ **minimale** identifizierende Attributmenge
Lokale Integritätsbedingung I	f: {r | r(R)} → Boolean mit r(R) = Relation über R Beispiel: Schlüsselbedingung für Attribut 1 • r_1: Attribut 1 enthält zwei gleiche Werte → false • r_2: Attribut 1 enthält nur unterschiedliche Werte → true
Erweitertes Relationenschema	$R_E = (R, I)$: Relationenschema plus lokale Integritätsbedingungen

DHBW Stuttgart

Schlüssel

- In relationalen Datenbanken werden alle **Beziehungen** über **Werte** hergestellt.
  - Dafür muss jedes Tupel einer Relation **eindeutig identifizierbar** sein.
  - Das leistet der **Schlüssel**.

Aussage	**Antwort**
Ein Schlüssel ist genau ein festgelegtes Attribut.	**Nein**, ein Schlüssel kann aus mehreren Attributen bestehen.
In einer Relation gibt es nur einen Schlüssel.	**Nein**, es kann mehrere geben; einer davon wird **Primärschlüssel**.
Ist $\{A_1, A_7, A_9\}$ ein Schlüssel, dann auch $\{A_1, A_7, A_9, A_{11}\}$?	**Nein**, es gilt das **Minimalitätsprinzip**: Mit A_{11} ist die Menge nicht mehr minimal, also nur ein Superschlüssel.

Schlüssel (Kandidatenschlüssel): minimale Menge von Attributen, deren Werte ein Tupel eindeutig identifizieren
Superschlüssel: jede Attributmenge, die Tupel eindeutig identifiziert (muss nicht minimal sein)
Primärschlüssel (PK): gewählter Kandidatenschlüssel; weitere heißen **Alternativschlüssel** (UNIQUE)
Der Primärschlüssel wird unterstrichen, z. B. Buch (ISBN, Titel, Autor)

DHBW Stuttgart

Integritätsbedingungen

Lokale Integrität	Bedingung innerhalb einer Relation, z. B. „Preis darf nicht negativ sein“. Erweitertes Relationenschema $R_E = (R, I)$: Für jede Bedingung $b \in I$ gilt $b(r) = true$.
Entitätsintegrität	Spezialfall der lokalen Integrität: Primärschlüssel-Attribute sind **eindeutig** und **NOT NULL**, es gibt keine zwei Tupel mit gleichem Schlüsselwert.
Globale Integrität	Bedingung über mehrere Relationen oder die gesamte Datenbank, z. B. die referenzielle Integrität.
Fremdschlüssel (FK)	Attributmenge, die auf den Primärschlüssel einer anderen Relation verweist. Relationale Datenbanken stellen **alle Beziehungen über Werte** her.
Vaterrelation	referenzierte Relation; enthält den Primärschlüssel, auf den verwiesen wird
Abhängige Relation	referenzierende Relation; enthält den Fremdschlüssel
Referenzielle Integrität	Jeder Fremdschlüsselwert muss als Schlüsselwert in der Vaterrelation vorkommen (oder NULL sein, falls erlaubt). Beispiel: Eine Rechnung darf nur auf eine KundenNr verweisen, die es in der Tabelle Kunde gibt.

Integritätsbedingungen (Vorschau)

Was passiert mit den abhängigen Tupeln, wenn ein Tupel der **Vaterrelation** gelöscht wird? Die Regel wird beim Fremdschlüssel festgelegt (ON DELETE …).

Regel	Wirkung beim Löschen
RESTRICT	Löschen wird abgelehnt, solange ein Fremdschlüssel auf das Tupel verweist. *Beispiel: Ein Kunde mit Rechnungen kann nicht gelöscht werden.*
CASCADE	Alle abhängigen Tupel werden mitgelöscht. *Beispiel: Mit dem Kunden werden auch seine Rechnungen gelöscht.*
SET NULL / SET DEFAULT	Die Fremdschlüssel der abhängigen Tupel werden auf NULL bzw. einen festgelegten Standardwert gesetzt. *Beispiel: Die Rechnung bleibt erhalten, KundenNr wird NULL.*

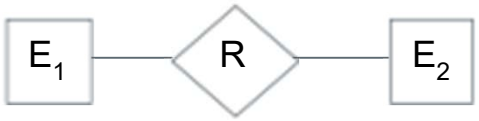
```
CREATE TABLE Kunde (
  KundeID INT PRIMARY KEY
);

CREATE TABLE Rechnung (
  RgID INT PRIMARY KEY,
  KundeID INT NOT NULL,
  CONSTRAINT FK_Rechnung_Kunde
    FOREIGN KEY (KundeID)
    REFERENCES Kunde(KundeID)
    ON DELETE RESTRICT
    ON UPDATE CASCADE
);
```

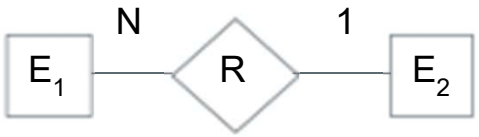
ON UPDATE regelt analog, was bei einer Änderung des Schlüssels passiert.

Wiederholung: Kardinalitäten

Standard ER-Modell

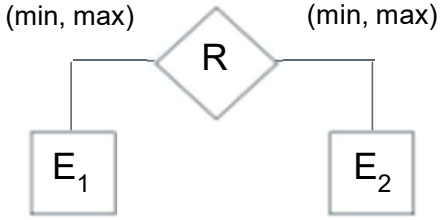


Standard ER-Modell
mit Kardinalitäten

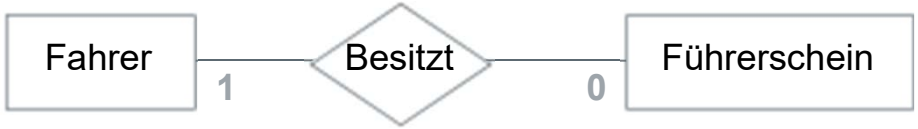


(min, max) Notation

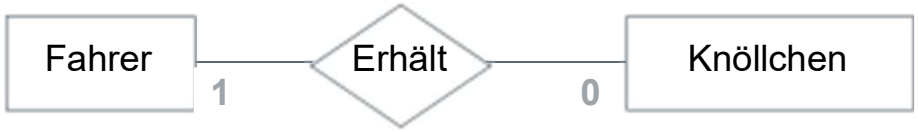
min: 0 = optional,
1 = Pflicht
max: höchstens 1 oder N



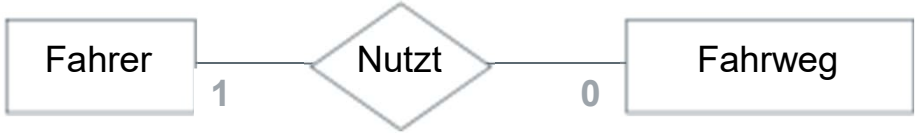
1 : 1



1 : N



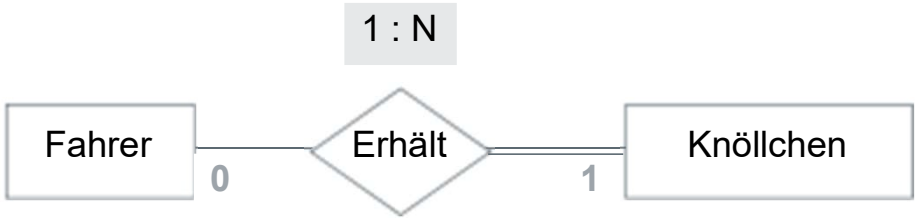
N : M

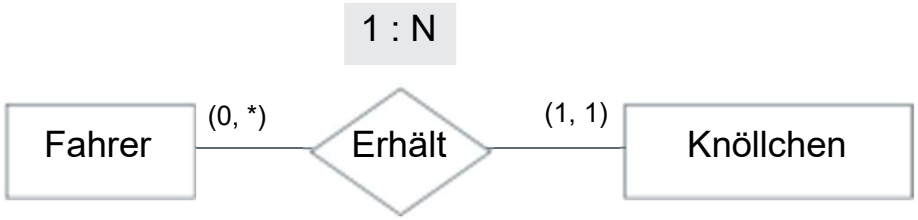


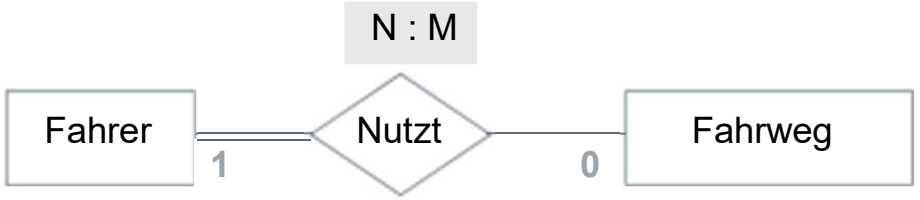
Wiederholung: Kardinalitäten

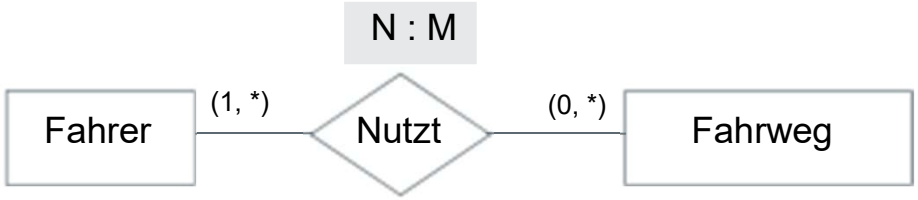








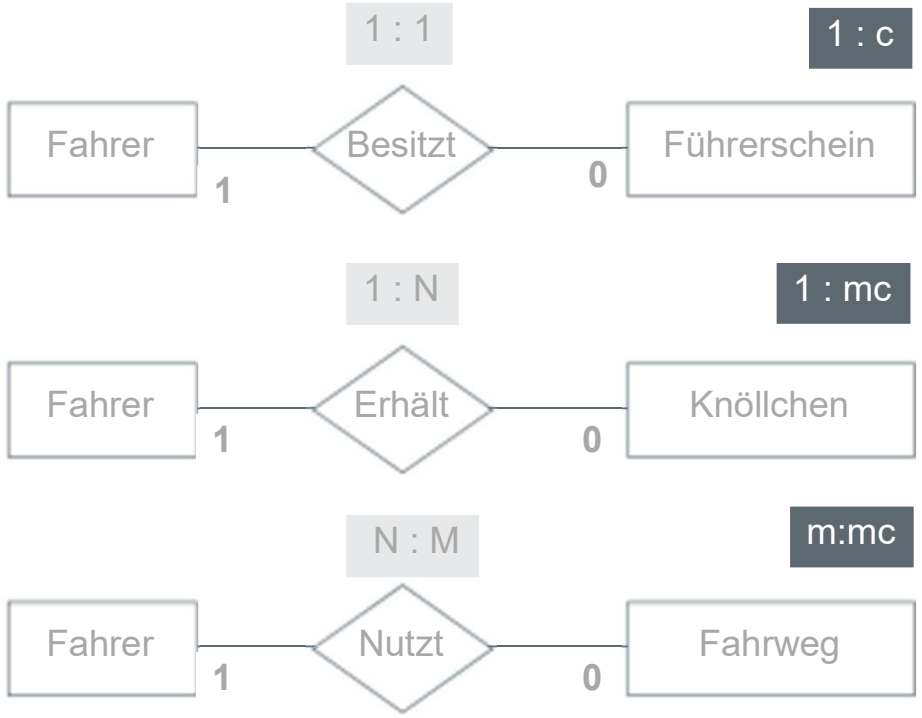




Mindestkardinalität: 0 = optional, 1 = Pflicht; die Doppellinie markiert die Pflichtteilnahme.

(min, max)-Notation: am Entitätstyp steht, wie oft er an der Beziehung teilnimmt; * = beliebig viele.

Typisierung der Assoziationen: Modifizierte Chen (MC)-Notation



Typ	Bezeichnung	Anzahl Tupel in der referenzierten Tabelle
1	einfach	genau ein Tupel
c	konditionell	kein oder ein Tupel
m	multipel	mindestens ein Tupel
mc	multipel-konditionell	beliebig viele (auch keins)

Beispiel 1 : c: Ein Führerschein gehört genau einem Fahrer (1); ein Fahrer hat keinen oder einen Führerschein (c).

Die MC-Notation ist eine feinere Typisierung. Für die Umsetzung ins Relationenmodell genügen die üblichen Kardinalitäten (1:1, 1:n, n:m) und die Platzierung der Fremdschlüssel.

Vom ER-Modell zum Relationenmodell

R2 \ R1	1	c	m	mc
1	1-1	c-1	m-1	mc-1
c	1-c	c-c	m-c	mc-c
m	1-m	c-m	m-m	mc-m
mc	1-mc	c-mc	m-mc	mc-mc

Hierarchisch: direkt über einen Fremdschlüssel abbildbar. Der FK steht auf der Seite mit „viele“, bei 1 : 1 zusätzlich UNIQUE.

Konditionell: mindestens eine Seite ist optional (c). Ein FK wäre oft NULL; Lösung über NULL-fähigen FK oder eine eigene Beziehungsrelation.

Netzwerkförmig: auf beiden Seiten mehrere (m/mc). Nur über eine Verknüpfungstabelle abbildbar.

Beziehungstyp: Er gibt an, wie viele Datensätze auf der einen Seite zu wie vielen auf der anderen gehören (z. B. 1 : m, jede Zelle der Matrix).

Das Relationenmodell kann **nur hierarchische Beziehungen direkt** abbilden. Konditionelle und netzwerkförmige Beziehungen werden mit **zusätzlichen Relationen** in hierarchische Beziehungen zerlegt.

Optionalität und Mindestanzahl (c/mc) regelt SQL über NULL / NOT NULL, UNIQUE und CHECK.

Überführung des ER-Modells ins Relationenmodell (1)

Vorgehen

Erst alles übersetzen (Initialentwurf), dann verfeinern

- **Entitäten übersetzen**: eine Relation pro Entitätstyp
- **Beziehungen übersetzen**: eine Relation pro Beziehung, egal welcher Typ
- **Schema verfeinern**: Relationen mit gleichem Schlüssel zusammenfassen, nur bei 1:1 und 1:m

Beispiel: Besitzt(ANr, PNr) und Auto(ANr, Modell), Schlüssel ANr → Auto(ANr, Modell, PNr)

Bei m:n nicht: Besitzt(PNr, ANr) bleibt getrennt.

Falsch bei 1:m: ANr in Person: Peter steht mit Manta und Golf zweimal drin, PNr ist kein Schlüssel.

Wer die Beziehungstypen sicher kennt, kann das Ergebnis direkt anlegen: s. Seiten 17 bis 21.

Falsche Modellierung erzeugt Redundanz

1 : m: Person (R_1) besitzt Autos (R_2)

Falsch: ANr in Person

PNr	Name	ANr
1	**Peter**	1
1	**Peter**	4
3	Gabi	2

Richtig: PNr in Auto

ANr	Name	PNr
1	Manta	1
4	Golf	1
2	C-Klasse	3

Peter steht zweimal in Person. Der Name ist doppelt, PNr ist kein Schlüssel mehr. Ändert man nur eine Zeile, widersprechen sich die Daten.

m : m: Golf gehört Peter und Gabi

Falsch: PNr in Auto

ANr	Name	PNr
4	**Golf**	1
4	**Golf**	3

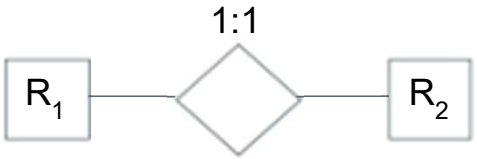
Richtig: Person_Auto

PNr	ANr
1	4
3	4

Der Golf steht zweimal in Auto, ANr ist kein Schlüssel mehr. Die Verknüpfungstabelle speichert jedes Paar genau einmal.

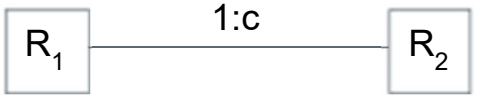
```sql
-- 1:1 durch UNIQUE absichern
CREATE TABLE Person (Pid INT PRIMARY KEY, ...);
CREATE TABLE Ausweis (
  Aid INT PRIMARY KEY,
  Pid INT UNIQUE,                    -- macht es 1:1
  CONSTRAINT FK_Ausweis_Person FOREIGN KEY (Pid) REFERENCES Person(Pid)
);
```

Ergebnis: hierarchisch



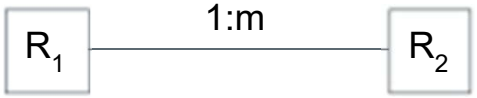
- **1 : 1**: Fremdschlüssel in einer der beiden Relationen (R_1 oder R_2), zusätzlich **UNIQUE**, damit jeder Wert nur einmal vorkommt (siehe Code oben). Alternative: beide Relationen zu einer zusammenfassen.

  Beispiel: `Ausweis(Aid, Pid): (10, 1), (11, 2); jede Pid nur einmal`



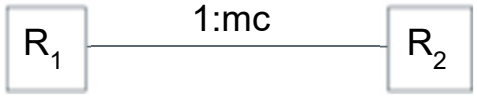
- **1 : c**: Fremdschlüssel in R_2, der auf R_1 verweist, ebenfalls UNIQUE. Beispiel: Ein Führerschein gehört genau einer Person; eine Person hat keinen oder einen Führerschein.

  Beispiel: `Führerschein(FNr, Pid): (7, 2); Pid 1 fehlt = kein Führerschein`



- **1 : m**: Fremdschlüssel in R_2, also auf der Seite mit „viele“. Beispiel: Eine Person (R_1) besitzt ein oder mehrere Autos (R_2); jedes Auto verweist auf seinen Besitzer. „Auto“ hat daher mindestens so viele Tupel wie „Person“.

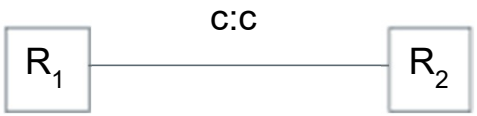
  Beispiel: `Auto(ANr, Pid): (1, 1), (4, 1), (2, 3); Person 1 hat zwei Autos`



- **1 : mc**: wie 1 : m, eine Person darf aber auch kein Auto haben.

  Beispiel: `Auto(ANr, Pid): (1, 1), (2, 3); Person 4 fehlt = kein Auto`

Konditionell: Problem mit Fremdschlüsseln



- **c : c**: Mögliche Lösung: Fremdschlüssel in R_1 und R_2, die jeweils auf die andere Relation verweisen (wie bei 1 : 1).
  - **Nachteil**: Beide Fremdschlüssel sind oft NULL, und jede Zuordnung steht doppelt.
  - Die beiden Seiten können sich widersprechen: Peter zeigt auf den Golf, der Golf auf Gabi.

Person (R_1)

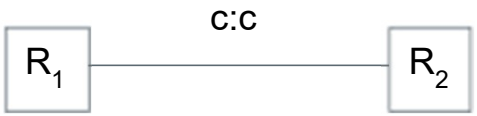
PNr	Name	ANr
1	Peter	**4**
3	Gabi	*NULL*

Auto (R_2)

ANr	Name	PNr
1	Manta	*NULL*
4	Golf	**3**

Rot: Peter fährt laut Person den Golf, laut Auto fährt ihn Gabi. NULL: kein Partner.

Ergebnis: konditionell



- **c : c**: Statt Fremdschlüsseln eine eigene **Beziehungsrelation** BR(#key1, #key2), die nur die tatsächlichen Zuordnungen enthält.
  - Keine NULL-Werte, jede Zuordnung steht genau einmal.
  - UNIQUE auf beiden Spalten sichert „höchstens eins“ auf jeder Seite.
- **c : m** und **c : mc**: analog über eine Beziehungsrelation.

Person (R_1)

PNr	Name
1	Peter
3	Gabi

Auto (R_2)

ANr	Name
1	Manta
4	Golf

BR

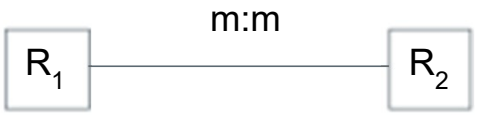
PNr	ANr
1	4

Beispiel

R_1 = Person, R_2 = Auto. Eine Person fährt kein oder ein Firmenauto; ein Auto wird von keiner oder einer Person gefahren.

Nur Peter fährt ein Auto (den Golf). Gabi und der Manta kommen in BR nicht vor.

Ergebnis: netzwerkförmig



- **m : m**: Eine Person kann ein oder mehrere Autos besitzen, und ein Auto kann einen oder mehrere Besitzer haben.
- **Was passiert mit einem Fremdschlüssel?** Steht PNr in „Autos“, muss ein Auto mit zwei Besitzern zweimal eingetragen werden (Golf): ANr ist **kein Schlüssel mehr**, und die Autodaten stehen **redundant** doppelt.
- **mc : m** und **mc : mc**: analog.

Personen

PNr	Name
1	Peter
2	Karl
3	Gabi
4	Monika

Autos

ANr	Name	PNr
1	Manta	1
2	C-Klasse	3
3	Uno	2
4	Golf	1
4	Golf	3
5	XJ12	4

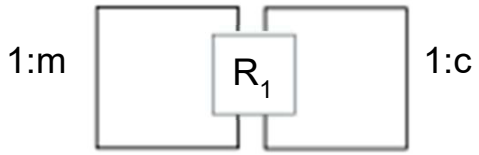
Golf doppelt: ANr ist nicht mehr eindeutig.

```
CREATE TABLE Person (PNr INT PRIMARY KEY, Name NVARCHAR(100));
CREATE TABLE Auto   (ANr INT PRIMARY KEY, Name NVARCHAR(100));

CREATE TABLE Person_Auto (
  PNr INT NOT NULL,
  ANr INT NOT NULL,
  PRIMARY KEY (PNr, ANr),
  FOREIGN KEY (PNr) REFERENCES Person(PNr),
  FOREIGN KEY (ANr) REFERENCES Auto(ANr)
);
```

Besser: Person_Auto(PNr, ANr) wie bei c : c; der PK verhindert doppelte Zuordnungen. Beispiel: (1, 4), (3, 4) = Golf bei Peter und Gabi.

Ergebnis: rekursiv (Sonderfall)



Rekursiv: Eine Relation steht in Beziehung zu sich selbst.

Beispiel: R_1 = Musiker. Ein Musiker kann Dirigent sein (1 : c), ein Dirigent dirigiert mehrere Musiker (1 : m). Der Fremdschlüssel DNr verweist auf MNr derselben Relation.

```
CREATE TABLE Musiker (
  MNr INT PRIMARY KEY,
  Name NVARCHAR(100),
  DNr INT NULL,  -- Dirigentennummer (optional)
  CONSTRAINT FK_Musiker_Dirigent
    FOREIGN KEY (DNr) REFERENCES Musiker(MNr)
);
```

Musiker

MNr	DNr	Name
1	2	Schmid
2	*NULL*	Karajan
3	*NULL*	Bernstein
4	2	Müller
5	3	Meier

→

Alternative

MNr	DNr	Name
1	1	Schmid
2	*NULL*	Karajan
3	*NULL*	Bernstein
4	1	Müller
5	2	Meier

Dirigent

DNr	MNr
1	2
2	3

- **Korrekt?** Ja, dieselben Beziehungen.
- **Ist Karajan Dirigent?** Links alle DNr nach MNr 2 durchsuchen; rechts genügt ein Blick in Dirigent.

DHBW Stuttgart

Überführung des ER-Modells ins Relationenmodell (2)

Der Initialentwurf (Seite 15 und 16) geht schematisch in drei Schritten vor:

1

Entitäten übersetzen

Eine Relation pro Entitätstyp. Die Attribute werden Spalten, der Schlüssel wird Primärschlüssel.

Beispiel: Person(PNr, Name)
Auto(ANr, Modell)

2

Beziehungen übersetzen

Eine Relation pro Beziehungstyp. Sie enthält die Schlüssel der beteiligten Entitäten als Fremdschlüssel.

Beispiel: Besitzt(PNr, ANr)

3

Schema verfeinern

Relationen mit gleichem Schlüssel zusammenfassen (Schlüsselkriterium), ggf. Schlüssel umbenennen.

Beispiel: Bei 1 : m hat Besitzt den Schlüssel ANr wie Auto → Auto(ANr, Modell, PNr)

Ergebnis: Bei 1 : m landet der Fremdschlüssel so automatisch auf der „viele“-Seite; bei m : m bleibt die Beziehungsrelation bestehen. Beide Optionen führen zum gleichen Schema.

Mögliche Umsetzung von Spezialisierungen (top-down)

Spezialisierung (top-down): Aus einem allgemeinen Entitätstyp (Flugzeug) werden speziellere Untertypen abgeleitet.

Umsetzung: Die Obertyp-Relation enthält die gemeinsamen Attribute. Jeder Untertyp erhält eine eigene Relation mit seinen speziellen Attributen.

FNr ist in den Untertyp-Relationen zugleich Primärschlüssel und Fremdschlüssel auf Flugzeug.

Das Attribut **Typ** zeigt, in welcher Untertyp-Relation die weiteren Daten stehen.

Flugzeug (Obertyp)

FNr	Typ	Baujahr
1	Segelflugzeug	2005
2	Passagierflugzeug	2010
3	Segelflugzeug	2001

Passagierflugzeug

FNr	Sitzplätze
2	150

Segelflugzeug

FNr	Spannweite (m)
1	8
3	10

FNr = Flugzeugnummer; unterstrichen = Primärschlüssel

DHBW Stuttgart

Mögliche Umsetzung von Generalisierungen (bottom-up)

Generalisierung (bottom-up) fasst mehrere konkrete Typen zu einem Obertyp zusammen (Segelflugzeug, Passagierflugzeug → Flugzeug). Für die Umsetzung gibt es drei Strategien; meist wird eine Obertyp-Tabelle genutzt, die mit jeder Subtyp-Tabelle 1 : c verknüpft ist.

Strategie	Umsetzung und Folgen
Single Table Table-per-Hierarchy	Eine Tabelle für alle Typen mit Diskriminator-Spalte Typ. Einfach, aber viele NULL-Werte.
Class Table Table-per-Type (TPT)	Obertyp-Tabelle plus eine Tabelle je Subtyp mit PK = FK. Ohne Redundanz, Abfragen brauchen Joins.
Concrete Table Table-per-Concrete-Class	Nur Subtyp-Tabellen; die Obertyp-Attribute stehen in jeder. Keine Joins, aber Redundanz.

Merke: Generalisierung und Spezialisierung sind zwei Blickrichtungen auf dieselbe Struktur. Die Fachabteilung denkt meist top-down, das DB-Team oft bottom-up.

Beispiel TPT: Segelflugzeug

FNr	Spannweite (m)
1	8
3	10
4	4
6	22

Jedes Segelflugzeug erbt seine Identität (FNr) von Flugzeug; nur die speziellen Attribute wie Spannweite stehen in der Subtyp-Tabelle.

Vielen Dank für Ihre Aufmerksamkeit!